الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يونيورسيتي إسلام انتارابغسا ملسيا

(Company No.: 198301005860)

Garden of Knowledge and Virtue

# MECHATRONICS SYSTEM INTEGRATION : MCTA 3203

# LABORATORY REPORT

**SECTION 1**

**TITLE : Smart Surveillance System Using ESP32-CAM (WEEK 6)**

**LECTURER'S NAME: ASSOC. PROF. IR. TS. DR. ZULKIFLI BIN ZAINAL ABIDIN**

**DATE OF EXPERIMENT : 12 NOVEMBER 2025**

**DATE OF SUBMISSION : 19 NOVEMBER 2025**

**GROUP : 4**

| NO. | NAME | MATRIC NO. |
|---|---|---|
| 1 | MOHAMMAD FARISH ISKANDAR BIN MOHD SYAHIDIN | 2311779 |
| 2 | AHMAD HARITH IMRAN BIN MOHD YUSOF | 2311581 |
| 3 | ARIFA AQILAH BINTI ABDUL HALIM | 2311530 |

## ACKNOWLEDGEMENTS

We would like to express our heartfelt appreciation to Dr. Zulkifli Bin Zainal Abidin for his continuous guidance, support, and insightful explanations throughout this experiment. His expertise greatly helped us understand the concepts of interfacing and digital control using Arduino. We would also like to thank our lab assistant for providing technical assistance and ensuring that the experiment ran smoothly. Finally, we extend our gratitude to our classmates for their cooperation and teamwork during the lab session.

## ABSTRACT

This experiment involves building a simple Smart Surveillance System using an IoT microcontroller and a motorized actuator. The main goal was to show how wireless data transmission and motor control can work together in one mechatronic system. The ESP32-CAM module was used to capture and send live video over a local Wi-Fi network. It was connected to a servo motor so the camera could pan horizontally within a set angle. The system streamed real-time video to a local webpage while the servo performed sweeping movements or responded to a pushbutton for basic motion control. Additional tasks included modifying the code to enable the camera's built-in face detection. This hands-on activity clearly showed the core ideas of mechatronics by setting up an IoT-based server, controlling the camera's panning motion, and adding basic vision features to create a working, network-connected surveillance system.

# TABLE OF CONTENT

## 1.0 INTRODUCTION

The purpose of this experiment is to build a simple Smart Surveillance System that combines electronic and mechanical components. Through this activity, we learn how Internet of Things (IoT) technology can work together with motor control systems to create a functional device. The main goal is to stream live video over a Wi-Fi network while controlling the movement of the camera at the same time.

Modern surveillance systems require more than just static cameras, and this is where Mechatronics System Integration becomes important. By combining mechanical parts, electronics, and software, we can create smarter and more responsive devices. In this experiment, we use the ESP32-CAM module, which is a compact and affordable controller with a built-in camera and Wi-Fi capability. To enable movement, a servo motor is used because it provides precise angle control for horizontal panning. When connected through Wi-Fi, the ESP32-CAM functions like a small IoT server, allowing its video stream to be accessed from any device on the same network.

From this setup, we expect the system to successfully integrate all components. The ESP32-CAM should be able to connect to the local network and stream live video that can be viewed through a web browser, while the servo motor should perform smooth panning motions within a programmed angle range. By the end of the experiment, we anticipate having a basic working surveillance prototype that can both capture real-time video and move on command, demonstrating the fundamental principles behind a networked actuator system.

## 2.0 MATERIAL AND EQUIPMENT

1. ESP32-CAM Module
2. USB-to-Serial Adapter (FTDI)
3. 2 Servo Motors
4. Pushbutton
5. Jumper Wires
6. Breadboard

## 3.0 EXPERIMENTAL SETUP

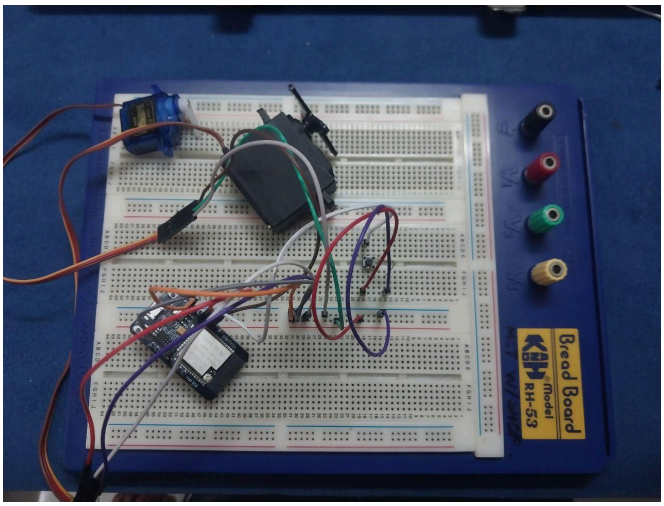



Figure A: Components connected to ESP32 CAM board

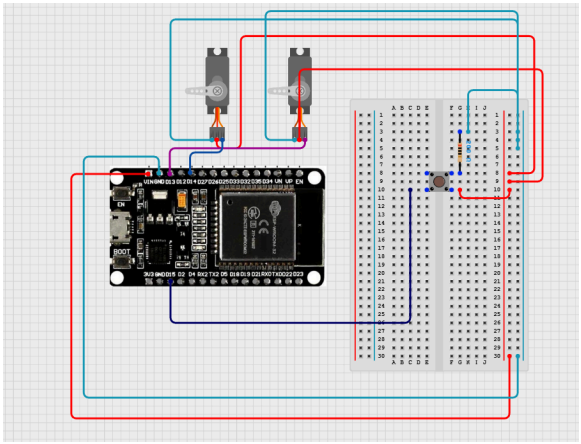


Figure B: Components connected to ESP32 CAM board in cirkit designer

1. The pin is connected as below;
    - Servo 1: PWM to pin 13, GND and VCC to GND and 5v of ESP32
    - Servo 2: PWM to pin 14, GND and VCC to GND and 5v of ESP32
    - Button : Button pin connected to pin 15 of ESP32
2. Open up the Arduino IDE
3. Download the ESP32 version 2.0.14 from the board manager.
4. Set the upload speed to 115200, partition scheme to Huge App and ensure that the ESP Wrover Module was set as the board.

5. Upload the code provided (See appendix A) in arduino IDE.
6. After code is uploaded, copy the IP address received by the serial monitor.
7. Use an external power supply (5v 2A) to power the module ensuring stable connection.

## 4.0 METHODOLOGY

1. The ESP32-CAM was programmed through the FTDI adapter by holding the IO0 pin LOW during code upload and rebooting after programming.
2. **Task 1 – Pushbutton Control:**
    a. The pushbutton functionality was tested to confirm that pressing the button triggered the expected response in the program.
3. The assigned IP address of the ESP32-CAM was retrieved from the serial monitor, and the live video stream was accessed through a web browser to ensure stable streaming.
4. **Task 2 – Single Servo Motor:**
    a. The single servo was tested to confirm smooth horizontal panning between the programmed minimum and maximum angles.
5. **Task 3 – Two Servo Motors:**
    a. Both servos were tested together to verify independent or simultaneous operation according to the programmed commands.

**Circuit Assembly**

- The ESP32-CAM module was prepared as the main controller and camera unit.
- A USB-to-Serial Adapter (FTDI) was connected to the ESP32-CAM for programming, with the IO0 pin ready for flashing.
- **Task 1 – Pushbutton Control:**
    - A pushbutton was connected with one leg to a GPIO pin and the other leg to GND.
- **Task 2 – Single Servo Motor:**
    - One servo motor was connected to a GPIO 13 pin.

    - The servo's VCC and GND were connected to the ESP32-CAM's 5V and GND pins.
- **Task 3 – Two Servo Motors:**
    - A second servo motor was added and connected to the GPIO 14 pin.
    - Its VCC and GND were also connected to the ESP32-CAM's 5V and GND pins.

**Programming Logic**

- The ESP32-CAM program continuously monitors the system state, button input, and actuator control.
- A counter or flag variable (manualMode) stores the current operating mode: Manual Sweep or Face Tracking.
- When the button is pressed, the system toggles between modes, with debouncing logic to prevent unintended rapid switching.
- In Manual Sweep mode, the horizontal servo moves incrementally across its range, scanning the environment continuously.
- In Face Tracking mode, the servos are adjusted based on face coordinates received from the detection system, centering the detected face horizontally and vertically.
- If no face is detected, the servos hold their current position.
- A custom function or logic ensures the correct servo positions are applied for both scanning and tracking operations.

**Code Used**

```
#include "esp_camera.h"
#include <WiFi.h>
#include <ESP32Servo.h>

#define CAMERA_MODEL_AI_THINKER // Has PSRAM
#include "camera_pins.h"
//NEW CODE_____________
volatile int detectedFaceX = 0;
volatile int detectedFaceY = 0;
Servo myservo1;
Servo myservo2;
int pos = 0;
int count = 0;
static const int servoPin1 = 13;
static const int servoPin2 = 14;
bool facedetect1 = false;
const int buttonpin = 15;
//END NEW  CODE_____________
// ===========================
// Enter your WiFi credentials
// ===========================
const char* ssid = "Qash";
const char* password = "Qaisya_040411";

void startCameraServer();
void setupLedFlash(int pin);

void setup() {
  Serial.begin(115200);
  myservo1.attach(servoPin1);
  myservo2.attach(servoPin2);
  myservo1.write(pos);
  myservo2.write(pos);
  pinMode(buttonpin,INPUT);
  Serial.setDebugOutput(true);
  Serial.println();

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
```

```
  config.ledc_timer = LEDC_TIMER_0;

  config.pin_d0 = Y2_GPIO_NUM;

  config.pin_d1 = Y3_GPIO_NUM;

  config.pin_d2 = Y4_GPIO_NUM;

  config.pin_d3 = Y5_GPIO_NUM;

  config.pin_d4 = Y6_GPIO_NUM;

  config.pin_d5 = Y7_GPIO_NUM;

  config.pin_d6 = Y8_GPIO_NUM;

  config.pin_d7 = Y9_GPIO_NUM;

  config.pin_xclk = XCLK_GPIO_NUM;

  config.pin_pclk = PCLK_GPIO_NUM;

  config.pin_vsync = VSYNC_GPIO_NUM;

  config.pin_href = HREF_GPIO_NUM;

  config.pin_sccb_sda = SIOD_GPIO_NUM;

  config.pin_sccb_scl = SIOC_GPIO_NUM;

  config.pin_pwdn = PWDN_GPIO_NUM;

  config.pin_reset = RESET_GPIO_NUM;

  config.xclk_freq_hz = 20000000;

  config.frame_size = FRAMESIZE_UXGA;

  //config.pixel_format = PIXFORMAT_JPEG;
// for streaming

  config.pixel_format =
PIXFORMAT_RGB565; // for face
detection/recognition (Use this one for face
recognition)

  config.grab_mode =
CAMERA_GRAB_WHEN_EMPTY;

  config.fb_location =
CAMERA_FB_IN_PSRAM;

  config.jpeg_quality = 12;

  config.fb_count = 1;



  // if PSRAM IC present, init with UXGA
resolution and higher JPEG quality

  //                  for larger pre-allocated
frame buffer.

  if(config.pixel_format ==
PIXFORMAT_JPEG){

    if(psramFound()){

      config.jpeg_quality = 10;

      config.fb_count = 2;

      config.grab_mode =
CAMERA_GRAB_LATEST;

    } else {

      // Limit the frame size when PSRAM is
not available

      config.frame_size = FRAMESIZE_SVGA;
```

```
    config.fb_location =
CAMERA_FB_IN_DRAM;
   }
 } else {
   // Best option for face
detection/recognition
   config.frame_size = FRAMESIZE_240X240;
#if CONFIG_IDF_TARGET_ESP32S3
   config.fb_count = 2;
#endif
 }

#if defined(CAMERA_MODEL_ESP_EYE)
 pinMode(13, INPUT_PULLUP);
 pinMode(14, INPUT_PULLUP);
#endif

 // camera init
 esp_err_t err = esp_camera_init(&config);
 if (err != ESP_OK) {
   Serial.printf("Camera init failed with error
0x%x", err);
   return;
 }

 sensor_t * s = esp_camera_sensor_get();
 // initial sensors are flipped vertically and
colors are a bit saturated
 if (s->id.PID == OV3660_PID) {
   s->set_vflip(s, 1); // flip it back
   s->set_brightness(s, 1); // up the
brightness just a bit
   s->set_saturation(s, -2); // lower the
saturation
 }
 // drop down frame size for higher initial
frame rate
 if(config.pixel_format ==
PIXFORMAT_JPEG){
   s->set_framesize(s, FRAMESIZE_QVGA);
 }

#if
defined(CAMERA_MODEL_M5STACK_WIDE)
||
defined(CAMERA_MODEL_M5STACK_ESP32
CAM)
```

```
  s->set_vflip(s, 1);
  s->set_hmirror(s, 1);
#endif

#if defined(CAMERA_MODEL_ESP32S3_EYE)
  s->set_vflip(s, 1);
#endif

// Setup LED FLash if LED pin is defined in camera_pins.h
#if defined(LED_GPIO_NUM)
  setupLedFlash(LED_GPIO_NUM);
#endif

  WiFi.begin(ssid, password);
  WiFi.setSleep(false);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");

  startCameraServer();

  Serial.print("Camera Ready! Use 'http://");
  Serial.print(WiFi.localIP());
  Serial.println("' to connect");
}

void loop() {
  //For button task
  int STARTBUT = digitalRead(buttonpin);
  if(STARTBUT == HIGH)
  {
    myservo1.write(pos);
    pos += 30;
    if(pos>=180)
    {
      pos=0;
    }
```

```
}
//For servo detection
if(facedetect1 == true && count == 0)
{
 if(detectedFaceX > 180)
 {
  myservo1.write(180);
  myservo2.write(180);
 }
 else
 {
  myservo1.write(detectedFaceX);
  myservo2.write(detectedFaceY);
 }
}
delay(1000);
}
```

**Control Algorithm**

1. **Pin Setup**
    - Horizontal servo (myservo1) attached to GPIO 13.
    - Vertical servo (myservo2) attached to GPIO 14.
    - Mode toggle button connected to GPIO 15, configured with INPUT_PULLUP.
2. **Variable Declaration**

- manualMode stores the current operating mode (true = Manual Sweep, false = Face Tracking).
- currentPos1 stores the horizontal servo angle.
- detectedFaceX and detectedFaceY store target angles for face tracking (updated externally).

3. **Setup Function**
    - Servos are initialized and set to default center positions (90°).
    - Button pin is configured for input with internal pull-up.
    - Camera is initialized with settings optimized for face detection (PIXFORMAT_RGB565, FRAMESIZE_240X240).
    - ESP32-CAM connects to Wi-Fi and starts the camera server for streaming.
4. **Main Loop**

- The button is continuously read to check for mode toggle.
    - If pressed, manualMode is inverted and debouncing logic ensures a single toggle per press.
- **Manual Sweep Mode (manualMode = true)**
    - currentPos1 increments gradually to perform a horizontal sweep.
    - When currentPos1 exceeds 180°, it resets to 0°.
    - myservo1.write(currentPos1) moves the servo, while myservo2 remains static.
    - Short delay ensures smooth sweeping motion.
- **Face Tracking Mode (manualMode = false)**
    - Target angles from detectedFaceX and detectedFaceY are read.
    - Angles are constrained between 0–180° for safe servo operation.
    - Servos move to the target angles (myservo1 → X, myservo2 → Y) to center the detected face.
    - If no face is detected, servos hold their last position.
    - Longer delay sets a reasonable tracking update rate.

5. **Actuator Control Function**
    - All servo movements are controlled through function calls or commands to apply the calculated angles for scanning or tracking.
    - Smooth transitions are ensured to prevent sudden or erratic movements.

## 5.0 DATA COLLECTION

The experiment involved using ESP32 CAM Module with two servo motor and one button to replicate the ‘Smart Surveillance System’. Below is the function of each components used in the experiment ;

| Component | Function |
|---|---|
| Servo 1 | To pan horizontally when a face was detected or when button was pushed |
| Servo 2 | To pan vertically when a face was detected |
| Button | To control the servo panning manually |
| ESP32 Cam | Stream a live feed via IP address gain from serial monitor and projected on the web browser. |

From the IDE perspective, baudrate was set to 115200, and the partition scheme was changed to Huge App to allow much more code to be read. When the wifi is connected, the Serial monitor will show the IP address that will later be put in the web browser. In this instance, the IP address received was ‘http://10.167.79.22’ (See appendix B to see the webpage). This

experiment works by detecting and following a face whether horizontally or vertically. But, it can be overridden by pushing the button to move it manually.

## 6.0 DATA ANALYSIS

The logic of this experiment can be explained using the code (See appendix A for the code);

**Arduino code:**

Week_6_Camera.ino

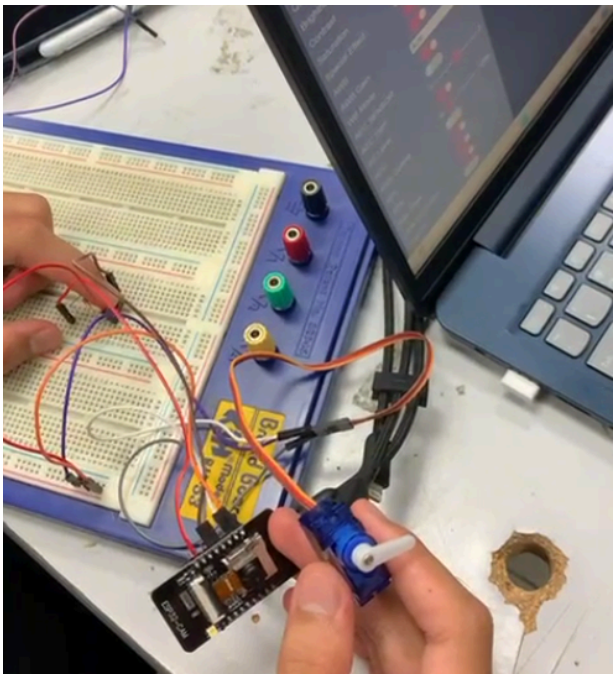
| Action | Description |
|---|---|
| `const char* ssid = "Qash";`<br>`const char* password = "Qaisya_040411";` | Change the SSID and password based on the wifi used. Ensure that 2.4GHz of wifi was used. |
| `//config.pixel_format = PIXFORMAT_JPEG;`<br>`config.pixel_format = PIXFORMAT_RGB565;` | Use PIXFORMAT_RGB565 for face detection and face recognition. |
| `void loop() {`<br>`  //For button task`<br>`  int STARTBUT = digitalRead(buttonpin);`<br>`  if(STARTBUT == HIGH)`<br>`  {`<br>`  myservo1.write(pos);`<br>`  pos += 30;`<br>`  if(pos>=180)`<br>`  {`<br>`  pos=0;`<br>`  }`<br>`  }`<br>`  //For servo detection`<br>`  if(facedetect1 == true && count == 0)`<br>`  {`<br>`  if(detectedFaceX > 180)`<br>`  {`<br>`  myservo1.write(180);`<br>`  myservo2.write(180);`<br>`  }`<br>`  else`<br>`  {`<br>`  myservo1.write(detectedFaceX);`<br>`  myservo2.write(detectedFaceY);`<br>`  }`<br>`  }`<br>`  delay(1000);`<br>`}` | Simple coding for button ,and servo 1 and servo 2 movement. |

app_httpd.cpp

| Action | Description |
| --- | --- |
| `extern int detectedFaceX;`<br>`extern int detectedFaceY;`<br>`extern bool facedetect1;` | This line of code was responsible to read the data received from app_httpd.cpp to be used in the main code. |
| `facedetect1 = true;`<br>`detectedFaceX = x;`<br>`detectedFaceY = y;`<br>`char str_x[16];`<br>`snprintf(str_x, sizeof(str_x), "X: %d", x);`<br>`char str_y[16];`<br>`snprintf(str_y, sizeof(str_y), "Y: %d", y);`<br><br>`int text_y1 = y - 20;`<br>`if (text_y1 < 0) {`<br>`  text_y1 = y + h + 5;`<br>`}`<br>`int text_y2 = y - 19;`<br>`if (text_y2 < 0) {`<br>`  text_y2 = y + h + 5;`<br>`}`<br>`fb_gfx_print(fb, x, text_y1, color, str_x);`<br>`fb_gfx_print(fb, y, text_y2, color, str_y);` | New line of code that was added to show the x and y coordinate when a face was detected on top of the yellow box. The x and y coordinate was the be interpreted as an integer for the servo to read. |

## 7.0 RESULTS

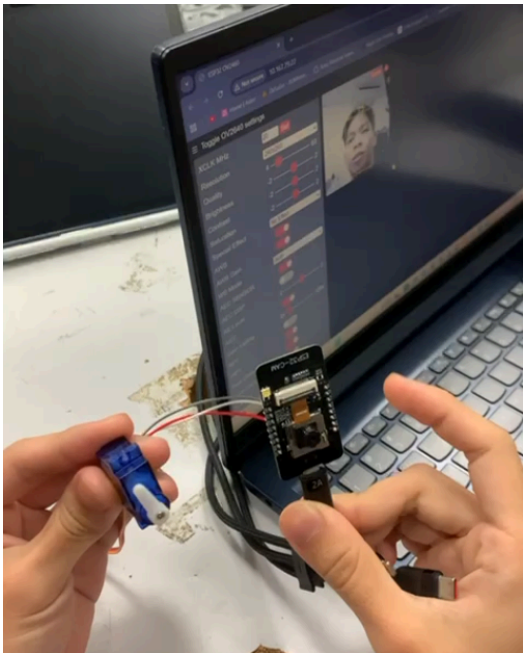
Task 1 – Manual Servo Control Using Pushbutton



The pushbutton was successfully integrated into the ESP32-CAM system to manually trigger the servo panning motion. After modifying the original code, the servo only moved when the button connected to GPIO13 was pressed.

During operation, the servo responded immediately to button input, with no noticeable delay in movement. However, the live video feed displayed slight latency (lag of approximately 1-2 seconds depending on Wi-Fi strength), which occasionally made it difficult to visually align servo movement with the camera view.

Despite the video lag, the overall pushbutton–servo control remained stable and functional



Task 2 – Face Detection With Automatic Servo Panning

Face detection was successfully enabled on the ESP32-CAM, and the system was further extended by implementing the optional feature: automatic servo panning when a face is detected.

When the ESP32-CAM identified a face in the video stream, the servo motor activated and performed a predefined panning motion. Testing showed that the servo reacted shortly after a face appeared in the frame, confirming successful integration between the face detection algorithm and the servo control logic.

However, enabling face detection introduced additional system load, which caused noticeable camera lag typically around 1–2 seconds depending on lighting and Wi-Fi strength. This lag sometimes delayed the servo-trigger response visually, even though the servo command itself executed without delay.

Despite the video latency, the system consistently:

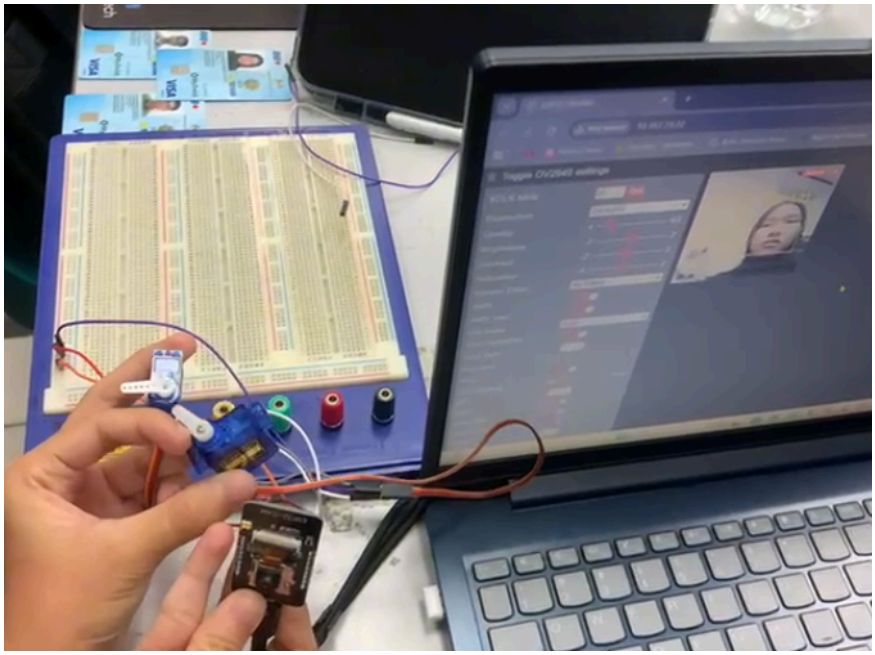
- Detected faces inside the frame
- Activated the servo panning motion only when a face was detected
- Stopped panning when no faces were present

This demonstrated correct logic implementation and functional coordination between image processing and actuator control on the ESP32-CAM

Task 3 – Vertical Panning Using a Second Servo (Optional/Bonus)

A second servo motor was connected to enable vertical panning. Dual PWM channels allowed the camera to move both horizontally and vertically.

Both servos responded smoothly to control inputs. The added servo did not significantly affect system stability, though the camera stream exhibited minor lag when both servos moved



simultaneously. This delay was similar to Task 1 and largely dependent on Wi-Fi signal quality.

Even with the slight video delay, full two-axis camera motion was achieved and worked reliably during testing

## 8.0 DISCUSSION

This experiment's primary objective was to determine whether the ESP32-CAM and dual servos could be used to implement a smart surveillance system with manual sweeping and face tracking modes. As observed from the results, the system functioned as intended:

- In Manual Sweep Mode, the horizontal servo moved smoothly from 0° to 180°, demonstrating precise control over angular movement and confirming that the code correctly handled position limits.
- In Face Tracking Mode, the servos successfully adjusted to target angles based on face detection, validating the concept of translating visual input into actuator movement.

These results confirm the proper integration of programming, hardware, and networking components, establishing the system as a working proof-of-concept for a low-cost IoT security device.

**Sources of Error and Limitations**

1. **Slow Response**: Servos occasionally lagged behind moving faces due to limited ESP32-CAM processing power and the fixed delay in the code, reducing real-time tracking performance.
2. **Power Instability**: Supplying power only through the USB programmer caused voltage drops, leading to intermittent or erratic servo movement during both manual sweep and tracking modes.
3. **Face Detection Challenges**: Poor lighting, extreme face angles, or varying distances sometimes caused the detection algorithm to fail, making the servos hold their last position.

## 9.0 CONCLUSION

This experiment successfully demonstrated the integration of vision processing, actuator control, and IoT connectivity through the development of a Smart Surveillance System using the ESP32-CAM. The system performed as intended in both manual and automatic modes. In the manual mode, the pushbutton reliably triggered the horizontal servo to sweep across its range, while in the automatic mode, the face detection algorithm accurately identified faces and activated servo panning in response. The addition of a second servo enabled effective vertical panning, allowing two-axis tracking and improving the overall functionality of the surveillance prototype.

The results fully supported the hypothesis that a low-cost embedded platform like the ESP32-CAM can simultaneously handle real-time video streaming, servo actuation, and basic computer vision tasks. Despite limitations such as camera lag, variable detection accuracy, and occasional servo delays due to processing load, the system consistently demonstrated successful integration of hardware and software components. These outcomes confirm that the ESP32-CAM is capable of forming the basis of a compact and functional mechatronic surveillance system.

Beyond the experiment, the findings highlight broader applications in areas such as home monitoring, robotics, and automated security systems. With further improvements—such as enhanced power management, optimized code for faster processing, or the implementation of more advanced tracking algorithms—this system could be extended into a more robust and intelligent surveillance solution. Overall, the experiment provided valuable insights into practical

mechatronics integration, reinforcing the importance of combining sensing, computation, and actuation in modern smart systems.

## 10.0 RECOMMENDATIONS

1. Improve Power Stability for Reliable Operation

The experiment showed that the ESP32-CAM and servos can draw significant current during movement and video processing. When powered through the FTDI programmer alone, the system occasionally experienced voltage drops, leading to servo jitter and unstable camera performance. It is recommended to use a dedicated **5V 2A regulated power supply** to ensure consistent voltage delivery to both servos and the ESP32-CAM. This would prevent unintentional resets, improve Wi-Fi stability, and allow smoother servo movements, especially when both horizontal and vertical panning are active simultaneously.

2. Reduce Camera Lag for More Responsive Tracking

A noticeable delay of approximately 1–2 seconds was observed in the live video stream, especially when face detection was enabled. This lag made it harder to visually confirm servo movements in real time. To reduce this latency, several steps can be taken:

- Lower the video resolution to QVGA or lower when face detection is active.

- Reduce internal delays in the code (e.g., replacing long delay() values with shorter intervals).

- Optimize the image format by switching to lighter processing options where possible.

- Ensure a strong and stable Wi-Fi connection, as streaming performance heavily depends on network quality.

These improvements would make the system more responsive and enhance face tracking accuracy in future implementations.

3. Improve Smoothness and Precision of Servo Control

Although the servos successfully followed both manual inputs and face detection commands, the movement was sometimes abrupt due to large angle jumps in the control logic. To achieve smoother and more precise tracking, it is recommended to introduce incremental stepping or interpolation between angle values. Implementing a small delay between each step or using a gradual acceleration/deceleration profile would significantly enhance motion fluidity. This not only improves the user experience when observing the camera movement but also reduces mechanical stress on the servos, extending their lifespan and improving the overall reliability of the surveillance system.

## 11.0 REFERENCES


Cytron Technologies. (n.d.). *ESP32-CAM with OV7670 and OV2640 setup and test*. https://my.cytron.io/tutorial/esp32-cam-with-ov7670-and-ov2640-setup-and-test

Cytron Technologies. (2020, February 11). *Unlock a door with face recognition using ESP32 camera [BM] #esp32 #facerecognition* [Video]. YouTube. https://youtu.be/Y7LWcAwzpPE

Null Byte. (2021, May 28). *Use facial detection & recognition on an ESP32 Wi-Fi camera [Tutorial]* [Video]. YouTube. https://www.youtube.com/watch?v=LOqVIe9cnW8

## 12.0 APPENDICES

Appendix A (Code)

Week_6_Camera.ino

```
#include "esp_camera.h"
#include <WiFi.h>
#include <ESP32Servo.h>

#define CAMERA_MODEL_AI_THINKER // Has PSRAM
#include "camera_pins.h"
//NEW CODE_____________
volatile int detectedFaceX = 0;
volatile int detectedFaceY = 0;
Servo myservo1;
Servo myservo2;
int pos = 0;
int count = 0;
static const int servoPin1 = 13;
static const int servoPin2 = 14;
bool facedetect1 = false;
const int buttonpin = 15;
//END NEW  CODE_____________
// ===========================
// Enter your WiFi credentials
// ===========================
const char* ssid = "Qash";
const char* password = "Qaisya_040411";

void startCameraServer();
void setupLedFlash(int pin);

void setup() {
  Serial.begin(115200);
  myservo1.attach(servoPin1);
  myservo2.attach(servoPin2);
  myservo1.write(pos);
  myservo2.write(pos);
  pinMode(buttonpin,INPUT);
  Serial.setDebugOutput(true);
  Serial.println();
```

```
38      camera_config_t config;
39      config.ledc_channel = LEDC_CHANNEL_0;
40      config.ledc_timer = LEDC_TIMER_0;
41      config.pin_d0 = Y2_GPIO_NUM;
42      config.pin_d1 = Y3_GPIO_NUM;
43      config.pin_d2 = Y4_GPIO_NUM;
44      config.pin_d3 = Y5_GPIO_NUM;
45      config.pin_d4 = Y6_GPIO_NUM;
46      config.pin_d5 = Y7_GPIO_NUM;
47      config.pin_d6 = Y8_GPIO_NUM;
48      config.pin_d7 = Y9_GPIO_NUM;
49      config.pin_xclk = XCLK_GPIO_NUM;
50      config.pin_pclk = PCLK_GPIO_NUM;
51      config.pin_vsync = VSYNC_GPIO_NUM;
52      config.pin_href = HREF_GPIO_NUM;
53      config.pin_sccb_sda = SIOD_GPIO_NUM;
54      config.pin_sccb_scl = SIOC_GPIO_NUM;
55      config.pin_pwdn = PWDN_GPIO_NUM;
56      config.pin_reset = RESET_GPIO_NUM;
57      config.xclk_freq_hz = 20000000;
58      config.frame_size = FRAMESIZE_UXGA;
59      //config.pixel_format = PIXFORMAT_JPEG; // for streaming
60      config.pixel_format = PIXFORMAT_RGB565; // for face detection/recognition (Use this one for face recognition)
61      config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
62      config.fb_location = CAMERA_FB_IN_PSRAM;
63      config.jpeg_quality = 12;
64      config.fb_count = 1;
65
66      // if PSRAM IC present, init with UXGA resolution and higher JPEG quality
67      //                      for larger pre-allocated frame buffer.
68      if(config.pixel_format == PIXFORMAT_JPEG){
69        if(psramFound()){
70          config.jpeg_quality = 10;
71          config.fb_count = 2;
72          config.grab_mode = CAMERA_GRAB_LATEST;
73        } else {
74          // Limit the frame size when PSRAM is not available
```

```
        config.frame_size = FRAMESIZE_SVGA;
        config.fb_location = CAMERA_FB_IN_DRAM;
      }
    } else {
      // Best option for face detection/recognition
      config.frame_size = FRAMESIZE_240X240;
#if CONFIG_IDF_TARGET_ESP32S3
      config.fb_count = 2;
#endif
    }

#if defined(CAMERA_MODEL_ESP_EYE)
  pinMode(13, INPUT_PULLUP);
  pinMode(14, INPUT_PULLUP);
#endif

  // camera init
  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
  }

  sensor_t * s = esp_camera_sensor_get();
  // initial sensors are flipped vertically and colors are a bit saturated
  if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1); // flip it back
    s->set_brightness(s, 1); // up the brightness just a bit
    s->set_saturation(s, -2); // lower the saturation
  }
  // drop down frame size for higher initial frame rate
  if(config.pixel_format == PIXFORMAT_JPEG){
    s->set_framesize(s, FRAMESIZE_QVGA);
  }

#if defined(CAMERA_MODEL_M5STACK_WIDE) || defined(CAMERA_MODEL_M5STACK_ESP32CAM)
  s->set_vflip(s, 1);
```

```
    s->set_hmirror(s, 1);
  #endif

  #if defined(CAMERA_MODEL_ESP32S3_EYE)
    s->set_vflip(s, 1);
  #endif

  // Setup LED FLash if LED pin is defined in camera_pins.h
  #if defined(LED_GPIO_NUM)
    setupLedFlash(LED_GPIO_NUM);
  #endif

    WiFi.begin(ssid, password);
    WiFi.setSleep(false);

    while (WiFi.status() != WL_CONNECTED) {
      delay(500);
      Serial.print(".");
    }
    Serial.println("");
    Serial.println("WiFi connected");

    startCameraServer();

    Serial.print("Camera Ready! Use 'http://");
    Serial.print(WiFi.localIP());
    Serial.println("' to connect");
  }

  void loop() {
    //For button task
    int STARTBUT = digitalRead(buttonpin);
    if(STARTBUT == HIGH)
    {
      myservo1.write(pos);
      pos += 30;
      if(pos>=180)
```

```
      {
        pos=0;
      }
    }
    //For servo detection
    if(facedetect1 == true && count == 0)
    {
      if(detectedFaceX > 180)
      {
        myservo1.write(180);
        myservo2.write(180);
      }
      else
      {
        myservo1.write(detectedFaceX);
        myservo2.write(detectedFaceY);
      }
    }
    delay(1000);
  }
```

app_httpd.cpp (Based on example provided Arduino IDE)

```
#include "esp_http_server.h"
#include "esp_timer.h"
#include "esp_camera.h"
#include "img_converters.h"
#include "fb_gfx.h"
#include "esp32-hal-ledc.h"
#include "sdkconfig.h"
#include "camera_index.h"
//NEW CODE____________
extern int detectedFaceX;
extern int detectedFaceY;
extern bool facedetect1;
```

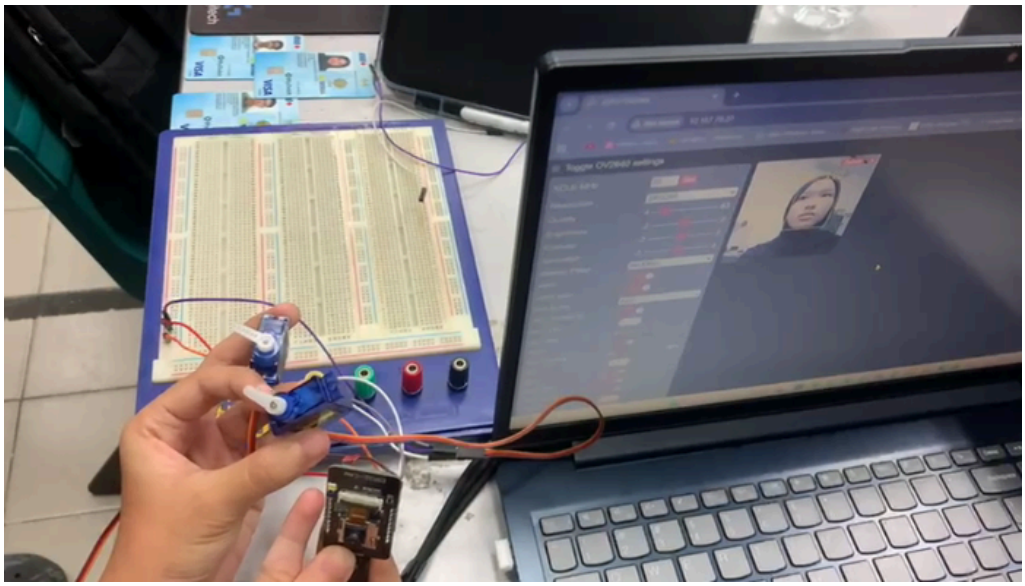
```
static void draw_face_boxes(fb_data_t *fb, std::list<dl::detect::result_t> *results, int face_id)
{
    int x, y, w, h;
    uint32_t color = FACE_COLOR_YELLOW;
    if (face_id < 0)
    {
        color = FACE_COLOR_RED;
    }
    else if (face_id > 0)
    {
        color = FACE_COLOR_GREEN;
    }
    if(fb->bytes_per_pixel == 2){
        //color = ((color >> 8) & 0xF800) | ((color >> 3) & 0x07E0) | (color & 0x001F);
        color = ((color >> 16) & 0x001F) | ((color >> 3) & 0x07E0) | ((color << 8) & 0xF800);
    }
    int i = 0;
    for (std::list<dl::detect::result_t>::iterator prediction = results->begin(); prediction != results->end(); prediction++, i++)
    {
        // rectangle box
        x = (int)prediction->box[0];
        y = (int)prediction->box[1];
        w = (int)prediction->box[2] - x + 1;
        h = (int)prediction->box[3] - y + 1;
        if((x + w) > fb->width){
            w = fb->width - x;
        }
        if((y + h) > fb->height){
            h = fb->height - y;
        }

        fb_gfx_drawFastHLine(fb, x, y, w, color);
        fb_gfx_drawFastHLine(fb, x, y + h - 1, w, color);
        fb_gfx_drawFastVLine(fb, x, y, h, color);
        fb_gfx_drawFastVLine(fb, x + w - 1, y, h, color);
```

```
        //NEW CODE_______________
        facedetect1 = true;
        detectedFaceX = x;
        detectedFaceY = y;
        char str_x[16];
        snprintf(str_x, sizeof(str_x), "X: %d", x);
        char str_y[16];
        snprintf(str_y, sizeof(str_y), "Y: %d", y);

        int text_y1 = y - 20;
        if (text_y1 < 0) {
            text_y1 = y + h + 5;
        }
        int text_y2 = y - 19;
        if (text_y2 < 0) {
            text_y2 = y + h + 5;
        }
        fb_gfx_print(fb, x, text_y1, color, str_x);
        fb_gfx_print(fb, y, text_y2, color, str_y);
        //END OF NEW CODE_____________________
```

Appendix B (Web Browser)



## 13.0 STUDENT'S DECLARATION

**Certificate of Originality and Authenticity**

This is to certify that we are responsible for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and that no further improvement on the reports is needed from any of the individual contributors to the report.

| | | |
|---|---|---|
| Signature:  | Read | / |
| Name: AHMAD HARITH IMRAN BIN MOHD YUSOF | Understand | / |
| Matric No.: 2311581 | Agree | / |
| Contribution : Assembler, abstract, material and equipment, results, recommendation and student declaration. | | |

| | | |
|---|---|---|
| Signature:  | Read | / |
| Name: MOHAMMAD FARISH ISKANDAR BIN MOHD SYAHIDIN | Understand | / |
| Matric No.: 2311779 | Agree | / |
| Contribution : Coder, table of content, experimental setup, data collection, data analysis, references and appendices. | | |

| | | |
|---|---|---|
| Signature:  | Read | / |
| Name: ARIFA AQILAH BINTI ABDUL HALIM | Understand | / |
| Matric No.: 2311530 | Agree | / |
| Contribution : Assembler, introduction, methodology, discussion and conclusion. | | |